Day-2: File Ownership and Access Control

Aditi Biju

261100690027

Part A — Ownership Investigation

1. Create a directory named ownership_lab

mkdir ownership_lab

2. Inside it, create the required files

cd ownership_lab

touch incident_report.txt network.log credentials.txt

3. Display detailed information for these files

ls -l

4. For each file, identify: Owner, Group, Owner permissions, Group permissions, Others permissions

- Owner: aduuu
- Group: aduuu
- Owner permissions: Read and Write
- Group permissions: Read and Write
- Others permissions: Read only

5. Determine UID and GID of the current user

id

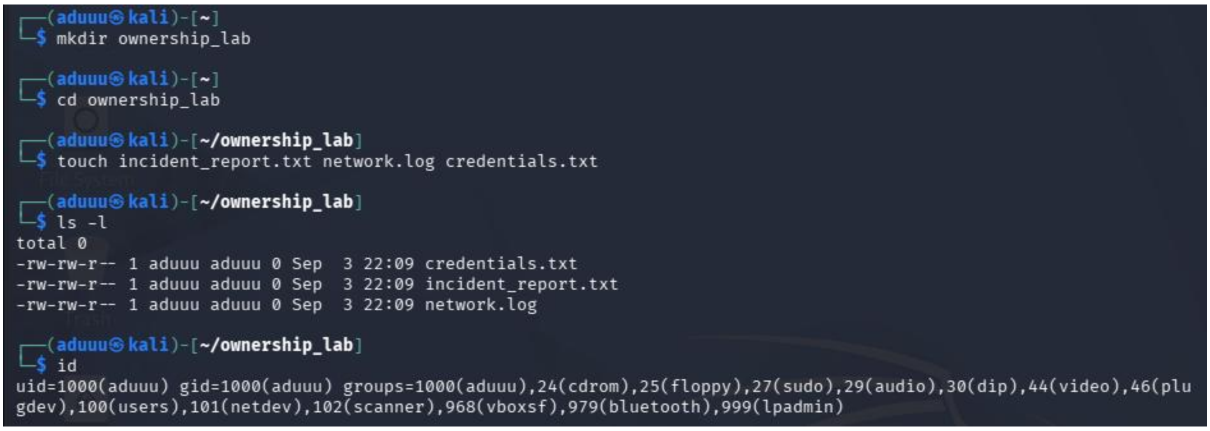
6. Compare the UID/GID information with the ownership displayed for the newly created files

Newly created files are owned by the user who creates them, and their group is usually the user's primary group. The id command shows the user's UID and GID, while ls -l shows the file's owner and group.

7. Difference between file ownership and file permissions

File Ownership: Answers the question "Who?" It identifies the user and group associated with a file.

File Permissions: Answers the question "What?" It determines what actions—Read, Write, and Execute—are allowed for the owner, group members, and other users.



Part B — chown: Changing Ownership

1. Change the owner of incident_report.txt to alice

sudo chown alice incident_report.txt

2. Change the owner of network.log to bob.

sudo chown bob network.log

3. Change the owner of credentials.txt to charlie.

sudo chown charlie credentials.txt

4. Verify each change.

ls -l

5. Try changing the owner of a file to another user without using sudo. Record the result

chown bob incident_report.txt

6. Explain why an ordinary user is generally not permitted to transfer file ownership to another user.

An ordinary user usually cannot transfer ownership of a file because it can create security problems. For example, someone could transfer a file to another user to avoid disk limits or assign a harmful file to someone else. Therefore, changing file ownership is normally limited to users with administrative privileges.

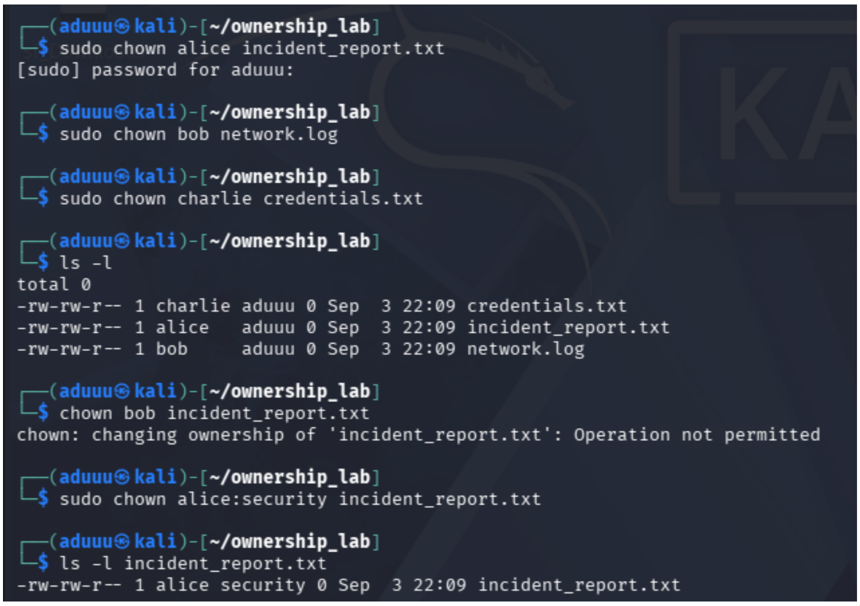
7. Using a single command, configure:

incident_report.txt

Owner : alice

Group : security

sudo chown alice:security incident_report.txt



```
┌──(aduuu㉿kali)-[~/ownership_lab]
└─$ sudo chown alice incident_report.txt
[sudo] password for aduuu:

┌──(aduuu㉿kali)-[~/ownership_lab]
└─$ sudo chown bob network.log

┌──(aduuu㉿kali)-[~/ownership_lab]
└─$ sudo chown charlie credentials.txt

┌──(aduuu㉿kali)-[~/ownership_lab]
└─$ ls -l
total 0
-rw-rw-r-- 1 charlie aduuu 0 Sep  3 22:09 credentials.txt
-rw-rw-r-- 1 alice   aduuu 0 Sep  3 22:09 incident_report.txt
-rw-rw-r-- 1 bob     aduuu 0 Sep  3 22:09 network.log

┌──(aduuu㉿kali)-[~/ownership_lab]
└─$ chown bob incident_report.txt
chown: changing ownership of 'incident_report.txt': Operation not permitted

┌──(aduuu㉿kali)-[~/ownership_lab]
└─$ sudo chown alice:security incident_report.txt

┌──(aduuu㉿kali)-[~/ownership_lab]
└─$ ls -l incident_report.txt
-rw-rw-r-- 1 alice security 0 Sep  3 22:09 incident_report.txt
```

Part C — chgrp: Changing Group Ownership

1. Change the group ownership of network.log to developers.

sudo chgrp developers network.log

2. Change the group ownership of credentials.txt to security.

sudo chgrp security credentials.txt

3. Verify the changes.

ls -l

4. Display the group memberships of alice, bob, and charlie.

groups alice

groups bob

groups charlie

5. Log in as alice and create alice_report.txt.

sudo su - alice

cd ownership_lab

touch alice_report.txt

6. Attempt to change the group of alice_report.txt to security.

chgrp security alice_report.txt

7. Attempt to change its group to developers.

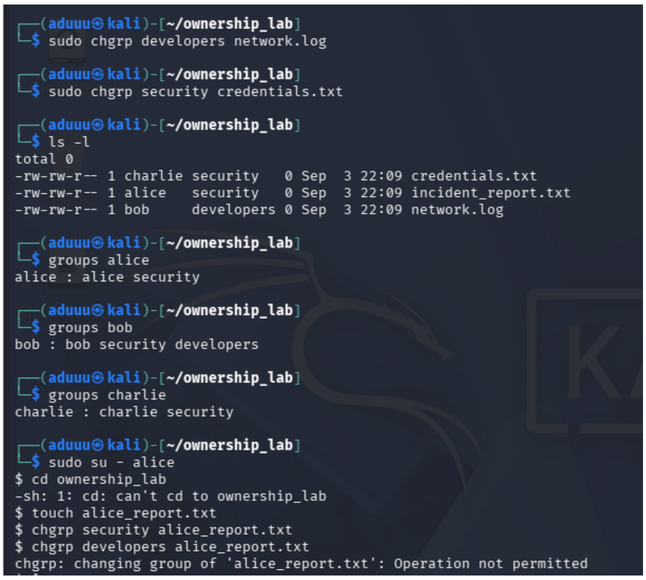
chgrp developers alice_report.txt

8. Compare the results.

chgrp: changing group of 'alice_report.txt': Operation not permitted

9. Explain how a user's group membership affects their ability to change the group ownership of a file they own.

A regular user can change the group of a file they own only to a group they are already a member of. So, their group membership decides which groups they can assign without using admin privileges.



Part D — Ownership + Permission Investigation

1. Configure -rw-r----- alice security incident_report.txt

sudo chmod 640 incident_report.txt

2. Identify what operations are available to: alice, charlie, bob

sudo usermod -aG security Charlie

3. A user who belongs to neither security nor the file owner charlie belongs to security. Record the result of each operation. Test whether Charlie can:

- Read incident_report.txt
- Modify incident_report.txt
- Delete incident_report.txt

sudo chmod 660 incident_report.txt

sudo su - charlie

cd ownership_lab

cat incident_report.txt

echo "test" >> incident_report.txt

rm incident_report.txt

4. Explain why Charlie's access changed even though the owner and group of the file did not change. Remove bob from the security group.

Charlie's access changed because the group permissions were changed from read-only (r--) to read and write (rw-). The group remained security, so Charlie, as a member of that group, automatically got write access

sudo gpasswd -d bob security

5. Explain why Bob's effective permissions changed.

Linux checks file permissions in this order: User → Group → Others.

When Bob was in the security group, he got the group permissions (rw-). After Bob was removed from the group, he no longer had group access, so Linux used the Others permissions (---), which means he had no access.